UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN**	**CICLO 02-2026**
	MATERIA: **DESARROLLO DE SOFTWARE EMPRESARIAL**	**GUIA DE LABORATORIO #7**
	PRÁCTICA: **API GATEWAY CON OCELOT**	

I. OBJETIVOS

- Que el estudiante configure una API Gateway utilizando Ocelot en un proyecto ASP.NET Core.
- Que el estudiante defina y configure rutas, agregación de servicios y políticas de seguridad en Ocelot.
- Que el estudiante despliegue la API Gateway en un entorno de desarrollo local y valide su funcionamiento integrando múltiples microservicios.

II. INTRODUCCIÓN TEORICA

¿Qué es una API Gateway?

Una API Gateway es un componente clave en la arquitectura de microservicios, actuando como un punto de entrada único para todas las solicitudes que llegan a un sistema distribuido de microservicios. En lugar de que los clientes interactúen directamente con los servicios individuales, todas las solicitudes pasan primero por la API Gateway, que luego las enruta al servicio adecuado, agregando, transformando o modificando las respuestas según sea necesario.

Beneficios de una API Gateway

- **Centralización del control**: permite centralizar aspectos como la autenticación, autorización, monitoreo y administración de tráfico, lo que simplifica la gestión y mejora la seguridad.
- **Transformación de datos**: facilita la transformación de formatos de datos entre el cliente y el microservicio, adaptando las necesidades del cliente sin afectar la implementación del servicio.
- **Balanceo de carga y tolerancia a fallos**: puede implementar mecanismos de balanceo de carga y circuit breakers para mejorar la disponibilidad y rendimiento del sistema.
- **Simplificación del cliente**: los clientes no necesitan conocer la estructura interna del sistema de microservicios, ya que la API Gateway maneja la complejidad de las interacciones.

Ventajas de una API Gateway

- **Reducción de la lógica en los clientes**: los clientes interactúan con una única API Gateway en lugar de múltiples servicios, reduciendo la complejidad y el acoplamiento.
- **Escalabilidad y mantenimiento**: permite escalar servicios de manera independiente y facilita el mantenimiento, ya que las modificaciones en la API Gateway no requieren cambios en los clientes.
- **Seguridad mejorada**: permite implementar políticas de seguridad centralizadas, reduciendo los riesgos y simplificando la gestión de accesos.

Funcionamiento de una API Gateway

Una API Gateway actúa como un intermediario entre los clientes y los microservicios que componen un sistema distribuido. Su funcionamiento se puede desglosar en varias etapas clave:

1. **Recepción de solicitudes**: el cliente realiza una solicitud HTTP o HTTPS, que es recibida por la API Gateway. Este cliente podría ser una aplicación móvil, un navegador web, u otro servicio que necesita interactuar con los microservicios.

2. **Autenticación y autorización**: la API Gateway verifica la autenticidad de la solicitud mediante mecanismos de autenticación, como tokens JWT, API Keys, o certificados. También se asegura de que el cliente tenga los permisos necesarios para acceder al recurso solicitado.
3. **Enrutamiento**: la API Gateway determina a qué microservicio debe enviar la solicitud basándose en la URL, encabezados, o datos del cuerpo de la solicitud. Este proceso de enrutamiento puede incluir la transformación de la solicitud, como la modificación de los parámetros o la conversión de formatos de datos.
4. **Agregación de servicios**: en algunos casos, la API Gateway puede necesitar llamar a múltiples microservicios para satisfacer una única solicitud del cliente. Agrega las respuestas de estos microservicios y las envía como una única respuesta al cliente.
5. **Transformación de respuestas**: antes de devolver la respuesta al cliente, la API Gateway puede realizar transformaciones, como cambiar el formato de los datos, agregar o quitar información, o comprimir la respuesta para optimizar la transmisión.
6. **Manejo de errores y retornos**: si uno de los microservicios devuelve un error o no está disponible, la API Gateway puede manejar estas situaciones aplicando políticas de tolerancia a fallos, como retries, circuit breakers, o retornando un mensaje de error estándar al cliente.
7. **Registro y monitoreo**: la API Gateway registra todas las solicitudes y respuestas, lo que permite monitorear el tráfico, identificar problemas de rendimiento, y mantener auditorías de acceso. Estos logs también son útiles para realizar análisis de seguridad.
8. **Implementación de políticas de seguridad**: la API Gateway puede aplicar políticas de seguridad como limitación de tasas (rate limiting), bloqueo de IPs sospechosas, y filtrado de solicitudes maliciosas, asegurando la protección de los microservicios detrás de ella.

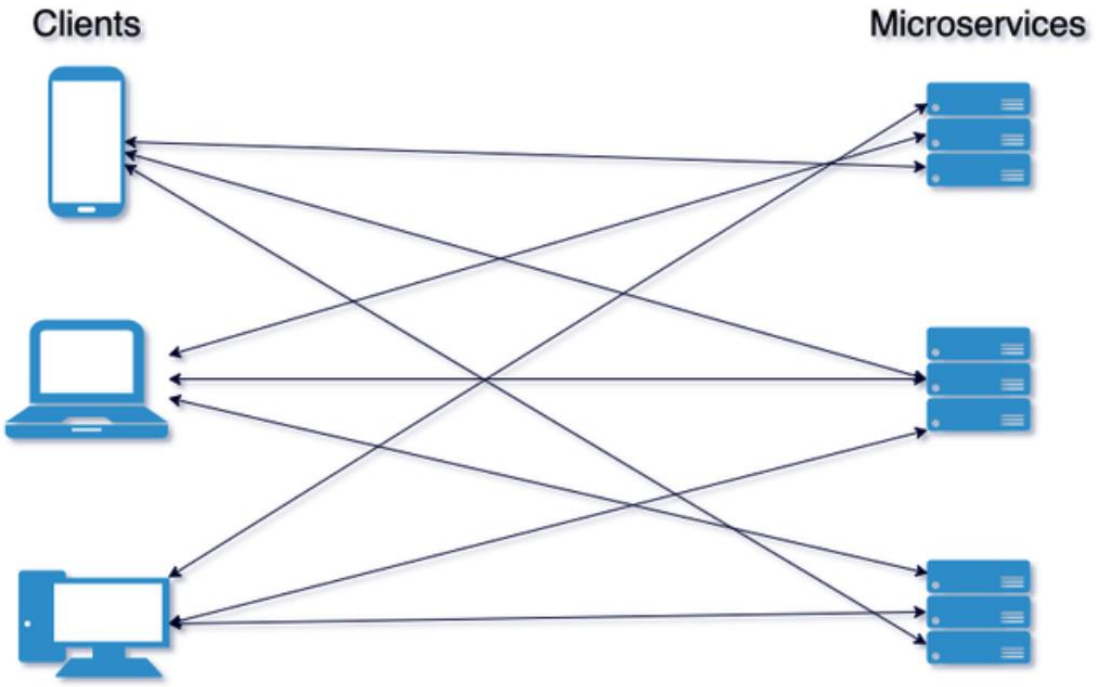


Ilustración 1 - Arquitectura sin API Gateway

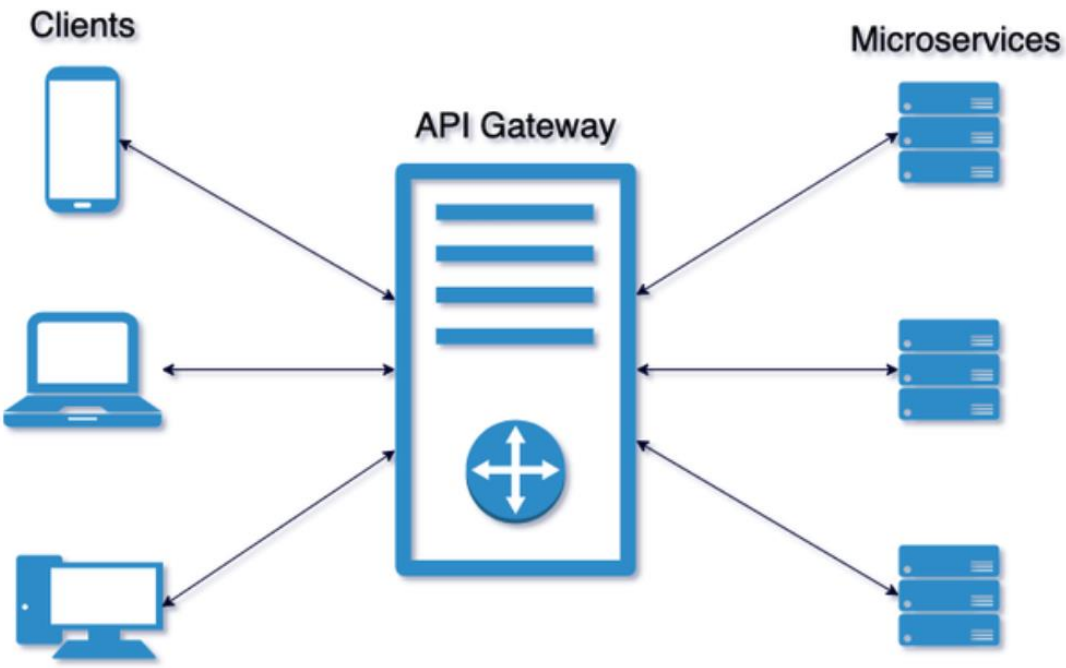


Ilustración 2 - Arquitectura con API Gateway

¿Qué es Ocelot?

Ocelot es un framework de API Gateway ligero y fácil de usar, diseñado específicamente para aplicaciones .NET, es popular en arquitecturas de microservicios basadas en .NET porque permite gestionar el enrutamiento, la autenticación, la autorización, la agregación de respuestas y la manipulación de datos de una manera sencilla y eficiente.

Beneficios de Ocelot

- **Integración con .NET Core**: se integra de manera nativa con aplicaciones ASP.NET Core, aprovechando el ecosistema de .NET para configuraciones, extensiones y servicios.
- **Facilidad de configuración**: ocelot se configura a través de archivos JSON, lo que facilita la definición de rutas, reglas de reenvío y políticas de manejo de solicitudes.
- **Flexibilidad y extensibilidad**: permite personalizar y extender su funcionalidad mediante el uso de middlewares de ASP.NET Core, lo que facilita la adaptación a necesidades específicas.

Ventajas de Ocelot

- **Rendimiento y ligereza**: al estar diseñado para ser ligero, Ocelot es ideal para escenarios donde se necesita un rendimiento óptimo con un consumo mínimo de recursos.
- **Soporte activo**: la comunidad de .NET apoya activamente a Ocelot, lo que garantiza actualizaciones frecuentes y soporte para nuevas versiones de .NET.
- **Versatilidad**: Ocelot permite manejar desde simples reenvíos de solicitudes hasta complejas configuraciones de balanceo de carga y políticas de autorización, haciéndolo adecuado para una variedad de escenarios.

Funcionamiento de Ocelot

Ocelot sigue un funcionamiento similar al de una API Gateway tradicional, pero con una implementación específica para el ecosistema .NET:

1. **Configuración inicial**: Ocelot se configura a través de un archivo JSON (ocelot.json), donde se definen rutas (Routes), reenvíos (Downstream), y políticas de manejo de solicitudes. Este archivo especifica cómo se deben mapear las solicitudes entrantes (Upstream) a los microservicios correspondientes.
2. **Pipeline de Middleware**: al estar basado en ASP.NET Core, Ocelot utiliza un pipeline de middleware donde cada solicitud pasa a través de una serie de componentes que pueden realizar operaciones como autenticación, autorización, transformación de datos, y enrutamiento.
3. **Enrutamiento dinámico**: Ocelot enruta las solicitudes entrantes a los microservicios adecuados basándose en las configuraciones de **ocelot.json**. Este enrutamiento puede ser dinámico, permitiendo la redirección de solicitudes a diferentes servicios dependiendo de condiciones como el contenido de los encabezados, la URL, o parámetros de la solicitud.
4. **Agregación de solicitudes**: Ocelot soporta la agregación de múltiples solicitudes en una sola, permitiendo que una solicitud del cliente desencadene varias solicitudes a diferentes microservicios, y luego combine las respuestas antes de devolverlas al cliente.
5. **Manejo de errores y retornos**: Ocelot maneja errores mediante configuraciones de retries, circuit breakers, y estrategias de fallback, asegurando que los clientes reciban respuestas consistentes incluso en situaciones de fallos parciales en los microservicios.
6. **Extensibilidad y personalización**: Ocelot permite la adición de middlewares personalizados dentro de su pipeline, lo que facilita la extensión de su funcionalidad para cumplir con requisitos específicos, como validaciones adicionales, modificaciones de encabezados, o implementaciones de políticas de seguridad personalizadas.

La función principal de Ocelot es recibir solicitudes HTTP entrantes y reenviarlas a un servicio de bajada. Actualmente, Ocelot solo admite esta función en forma de otra solicitud HTTP. Ocelot describe el enrutamiento de una solicitud a otra como una ruta. Para que algo funcione en Ocelot, es necesario configurar una ruta en la configuración.



1. **DownstreamPathTemplate**: especifica la plantilla de la ruta que se debe utilizar para las solicitudes que Ocelot enviará a los servicios downstream (los servicios reales que están detrás de la API Gateway).
    - **Ejemplo**: **"/api/productos/{id}"** puede mapearse a una API específica en el backend.

2. **UpstreamPathTemplate**: define la ruta en la API Gateway que los clientes deben usar para acceder a los servicios. Ocelot mapea esta ruta a la ruta DownstreamPathTemplate.
    - **Ejemplo**: **"/productos/{id}"** será la ruta que los clientes usarán para acceder al servicio.

3. **UpstreamHttpMethod**: lista los métodos HTTP que se permiten para esta ruta. Ejemplos comunes son GET, POST, PUT, DELETE.
    - **Ejemplo**: **["Get"]** significa que solo se permiten solicitudes GET en esta ruta.

4. **DownstreamHttpMethod**: define el método HTTP que se usará para las solicitudes downstream. Si no se especifica, se usará el mismo método que en UpstreamHttpMethod.
    - **Ejemplo**: **"POST"** para que Ocelot use POST al enviar la solicitud al servicio backend.

5. **DownstreamHttpVersion**: especifica la versión de HTTP que se usará para las solicitudes downstream (como 1.1 o 2.0).
    - **Ejemplo**: **"1.1"** para usar HTTP/1.1 en la comunicación con el servicio downstream.

6. **AddHeadersToRequest**: permite agregar encabezados adicionales a la solicitud antes de enviarla al servicio downstream.
    - **Ejemplo**: **{"X-Custom-Header": "value"}** agrega un encabezado personalizado.

7. **AddClaimsToRequest**: permite agregar claims (reclamaciones) a la solicitud antes de enviarla al servicio downstream.
    - **Ejemplo**: **{"UserId": "sub"}** agrega un claim basado en el claim sub del token.

8. **RouteClaimsRequirement**: define qué claims se requieren para que una solicitud coincida con esta ruta.
    - **Ejemplo**: **{"role": "admin"}** especifica que el usuario debe tener el rol de admin.

9. **AddQueriesToRequest**: permite agregar parámetros de consulta (query parameters) adicionales a la solicitud antes de enviarla al servicio downstream.
    - **Ejemplo**: **{"extraParam": "value"}** agrega un parámetro adicional a la URL.

10. **RequestIdKey**: define una clave para el identificador de la solicitud que se pasa a través de Ocelot, lo que permite rastrear la solicitud a lo largo del pipeline.
    - **Ejemplo**: **"X-Request-Id"** especifica que el ID de la solicitud debe ser añadido a los encabezados con esta clave.

11. **FileCacheOptions**: configura la caché basada en archivos. Esto permite a Ocelot almacenar en caché las respuestas para mejorar el rendimiento.
    - **Sub-Atributos**:
        i. TtlSeconds: Tiempo de vida en segundos de los elementos en la caché.
        ii. Region: Región de la caché utilizada (por ejemplo, europe-central).
    - **Ejemplo**: **{"TtlSeconds": 300, "Region": "us-east"}** para caché con TTL de 5 minutos.

12. **RouteIsCaseSensitive**: indica si la ruta debe ser sensible a mayúsculas y minúsculas.
    - **Ejemplo**: **false** significa que **/Productos** y **/productos** se tratarán como la misma ruta.

13. **ServiceName**: nombre del servicio que se utilizará con un sistema de descubrimiento de servicios como Consul o Eureka.
    - **Ejemplo**: **"ProductService"** para identificar el servicio específico en un entorno de microservicios.

14. **DownstreamScheme**: define el esquema a utilizar para las solicitudes downstream (http o https).
    - **Ejemplo**: **"https"** para usar HTTPS en la comunicación con el servicio downstream.

15. **DownstreamHostAndPorts**: lista de hosts y puertos a los que Ocelot debe dirigir las solicitudes.
    - **Ejemplo**:

```
[
    { "Host": "localhost", "Port": 5001 }
]
```

16. **QoSOptions**: configura opciones de calidad de servicio (QoS), como circuit breaker.
    - **Sub-Atributos**:
        i. **ExceptionsAllowedBeforeBreaking**: número de excepciones permitidas antes de activar el circuit breaker.
        ii. **DurationOfBreak**: duración del tiempo de espera cuando el circuit breaker está activado.
        iii. **TimeoutValue**: valor de tiempo de espera para las solicitudes.
    - **Ejemplo**:

```
{
    "ExceptionsAllowedBeforeBreaking": 3,
    "DurationOfBreak": 1000,
    "TimeoutValue": 5000
}
```

17. **LoadBalancer**: especifica el algoritmo de balanceo de carga que Ocelot debe usar, como RoundRobin, LeastConnection, etc.
    - **Ejemplo**: "RoundRobin" para distribuir equitativamente las solicitudes.

18. RateLimitOptions: Configura las opciones de limitación de tasa (rate limiting) para la ruta.
    - **Sub-Atributos**:
        i. **ClientWhitelist**: lista de clientes exentos de la limitación.
        ii. **EnableRateLimiting**: activa o desactiva la limitación de tasa.
        iii. **Period**: periodo de tiempo para el cálculo de la tasa ("1s", "1m", "1h").
        iv. **PeriodTimespan**: tiempo específico para el cálculo de la tasa en segundos.
        v. **Limit**: número máximo de solicitudes permitidas en el periodo especificado.
    - **Ejemplo**:

```
{
    "EnableRateLimiting": true,
    "Period": "1m",
    "Limit": 100
}
```

19. **AuthenticationOptions**: configura las opciones de autenticación para la ruta.
    - **Sub-Atributos**:
        i. **AuthenticationProviderKey**: clave del proveedor de autenticación (por ejemplo, IdentityServer).
        ii. **AllowedScopes**: lista de scopes permitidos.
    - **Ejemplo**:

```
{
    "AuthenticationProviderKey": "Bearer",
    "AllowedScopes": [
        "api1.read",
        "api1.write"
    ]
}
```

20. **HttpHandlerOptions**: configura opciones avanzadas para el manejo de solicitudes HTTP.
    - **Sub-Atributos**:
        i. **AllowAutoRedirect**: si se permiten redirecciones automáticas.
        ii. **UseCookieContainer**: si se deben usar contenedores de cookies.
        iii. **UseTracing**: si se debe habilitar el rastreo.
        iv. **MaxConnectionsPerServer**: máximo de conexiones permitidas por servidor.
    - **Ejemplo**:

```
{
    "AllowAutoRedirect": true,
    "UseCookieContainer": true,
    "UseTracing": false,
    "MaxConnectionsPerServer": 100
}
```

21. **DangerousAcceptAnyServerCertificateValidator**: cuando se establece en true, Ocelot acepta cualquier certificado de servidor, incluso si no es de confianza. Esto es peligroso y solo debe usarse en entornos de desarrollo.
    - **Ejemplo**: **false** para rechazar certificados no confiables.

22. **SecurityOptions**: configura opciones de seguridad, como la lista de IPs permitidas o bloqueadas.
    - **Sub-Atributos**:
        i. **IPAllowedList**: lista de direcciones IP permitidas.
        ii. **IPBlockedList**: lista de direcciones IP bloqueadas.
        iii. **ExcludeAllowedFromBlocked**: si las IP permitidas deben excluirse de las bloqueadas.
    - **Ejemplo**:

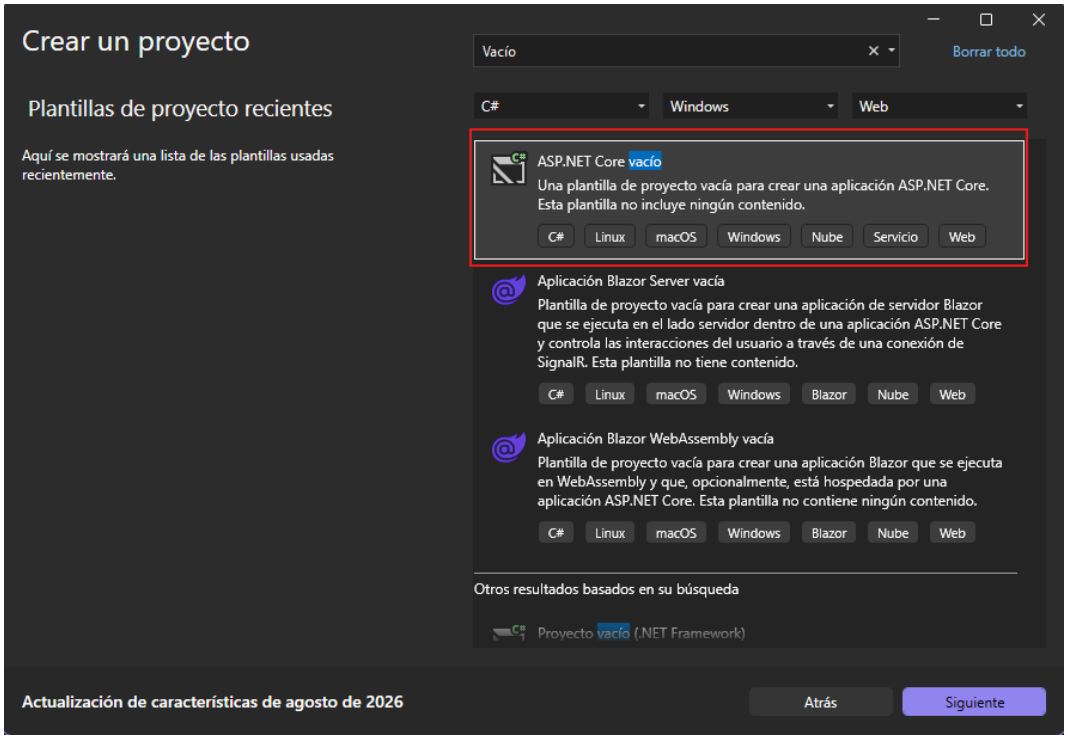
```
{
    "IPAllowedList": ["192.168.1.1"],
    "IPBlockedList": ["0.0.0.0"],
    "ExcludeAllowedFromBlocked": true
}
```

III. DESARROLLO DE LA PRÁCTICA

Parte 1 – Proyectos de ASP.NET Core Web API

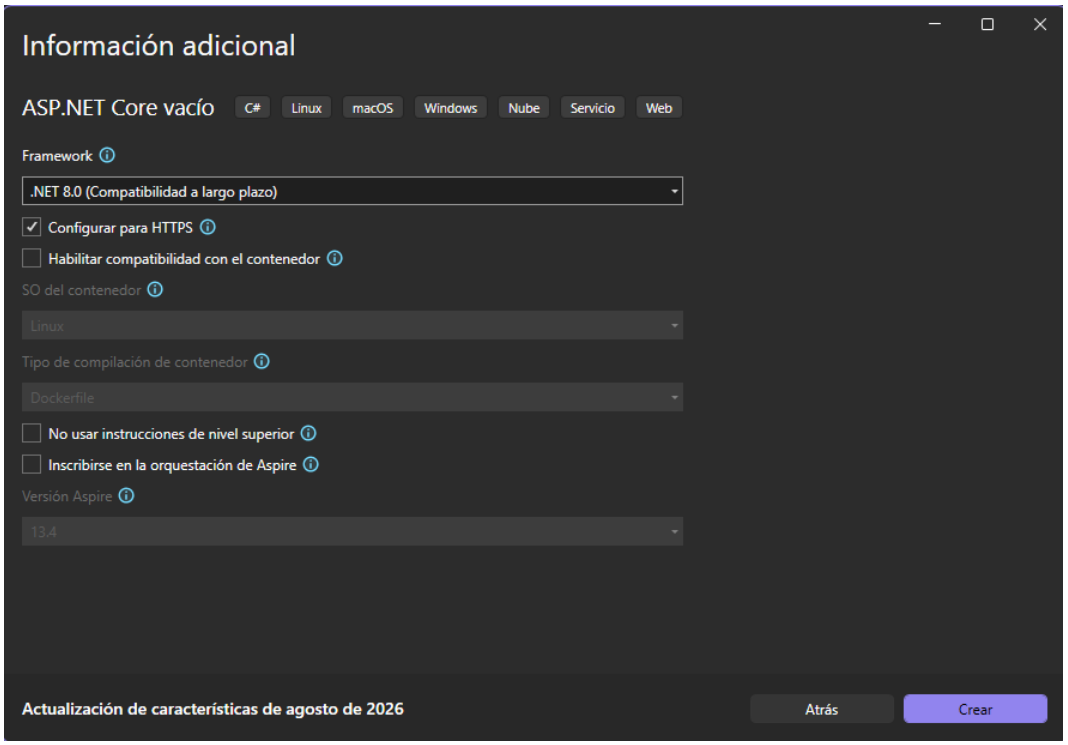
1. Abra Visual Studio y seleccione la opción **Crear un proyecto**.

2. Seleccione la plantilla **ASP.NET Core Empty**.

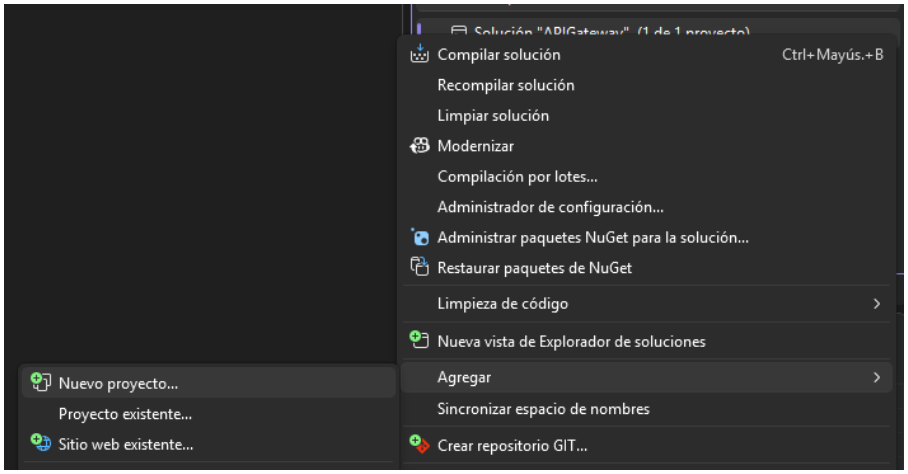


3. Asigne el nombre **APIGateway**.

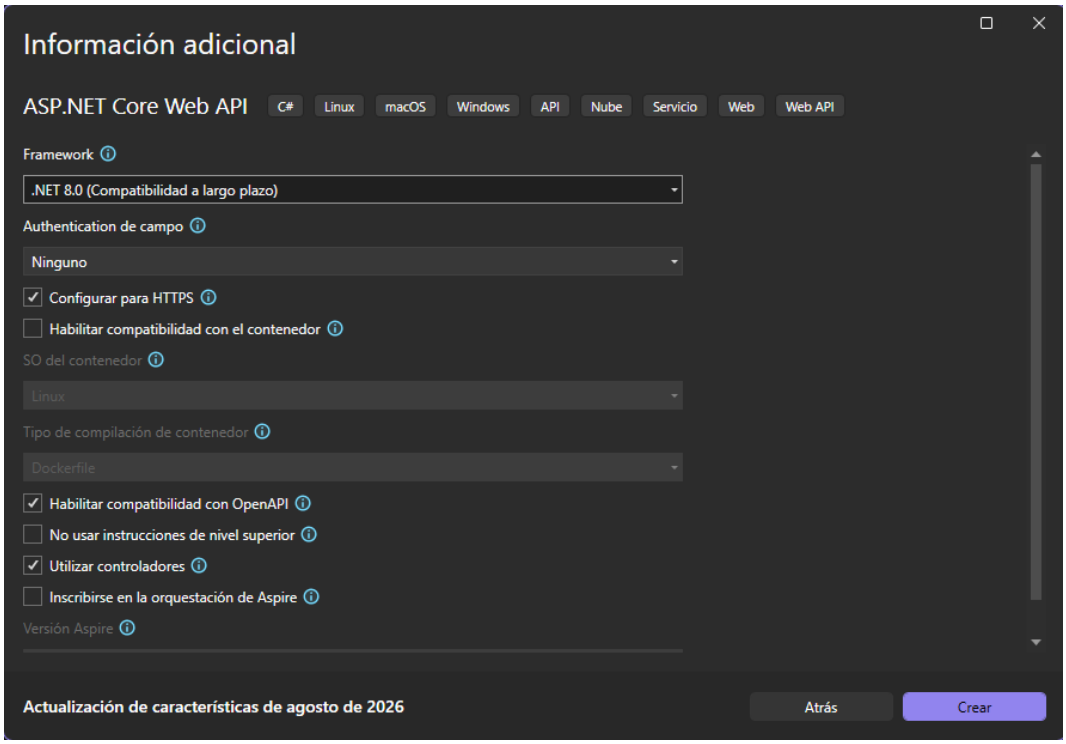
4. Seleccione el framework .NET 8 o superior y haga clic en **Crear**.



5. Cuando finalice la creación del proyecto, haga clic derecho sobre la solución y seleccione **Agregar > Nuevo proyecto**.

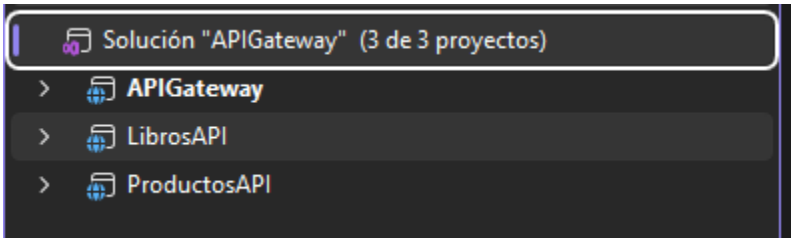


6. Seleccione la plantilla **ASP.NET Core Web API**, asigne el nombre al **ProductosAPI** y utilice la versión **.NET 8**.

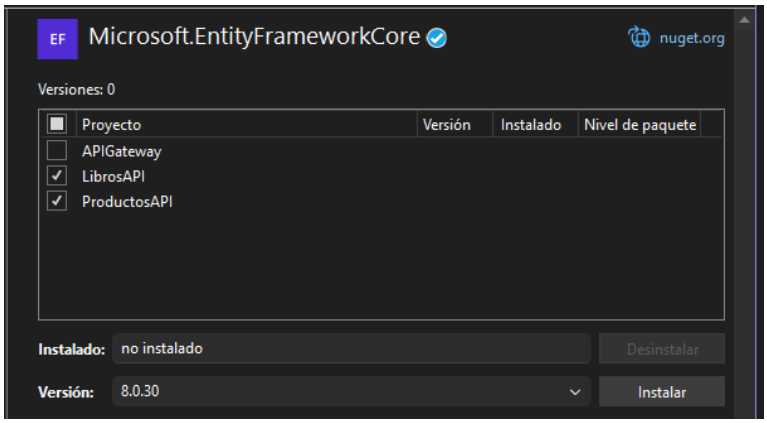


7. Repita los pasos y agregue otro proyecto **ASP.NET Core Web API** con el nombre **LibrosAPI**.

8. Una vez creados los proyectos, su solución debe visualizarse de la siguiente manera.



9. Abra el administrador de paquetes NuGet e instale el paquete **Microsoft.EntityFrameworkCore** en los proyectos **ProductosAPI** y **LibrosAPI**.



10. Realice los mismos pasos e instale los paquetes **Microsoft.EntityFrameworkCore.SqlServer** y **Microsoft.EntityFrameworkCore.Tool**.

Parte 2 – API de productos

1. Dentro del proyecto **ProductosAPI**, cree una nueva carpeta con el nombre **Models**.

2. Dentro de la carpeta **Models**, cree una nueva clase con el nombre **Producto** y agregue el siguiente código.

```
using System.ComponentModel.DataAnnotations;

namespace ProductosAPI.Models;

public class Producto
{
    public int Id { get; set; }

    [Required]
    public string Nombre { get; set; } = string.Empty;

    public string? Categoria{ get; set; }

    public string? Descripcion { get; set; }
}
```

3. Dentro de la carpeta **Models**, cree una nueva clase con el nombre **ProductosDbContext** y modifique el contenido como se muestra a continuación.

```
using Microsoft.EntityFrameworkCore;
using static System.Runtime.InteropServices.JavaScript.JSType;

namespace ProductosAPI.Models;

public class ProductosDbContext(DbContextOptions options) : DbContext(options)
{
    public DbSet<Producto> Productos { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
```

```
        modelBuilder.Entity<Producto>().HasData(
            new Producto
            {
                Id = 1,
                Nombre = "Laptop",
                Categoria = "Electrónica",
                Descripcion = "Laptop de alto rendimiento"
            },
            new Producto{
                Id = 2,
                Nombre = "Smartphone",
                Categoria = "Electrónica",
                Descripcion = "Smartphone de última generación"
            },
            new Producto {
                Id = 3,
                Nombre = "Silla de escritorio",
                Categoria = "Muebles",
                Descripcion = "Silla de escritorio ejecutivo"
            }
        );
    }
}
```

4. Abra el archivo **appsettings.json** y añada la cadena de conexión para su base de datos, puede utilizar una instancia distinta a la localdb.

```
{
  "ConnectionStrings": {
        "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=ProductosAPI;Trusted_Connection=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

5. Abra el archivo **Program.cs** y agregue el siguiente bloque de código para registrar el contexto de EntityFramework.

```
var builder = WebApplication.CreateBuilder(args);

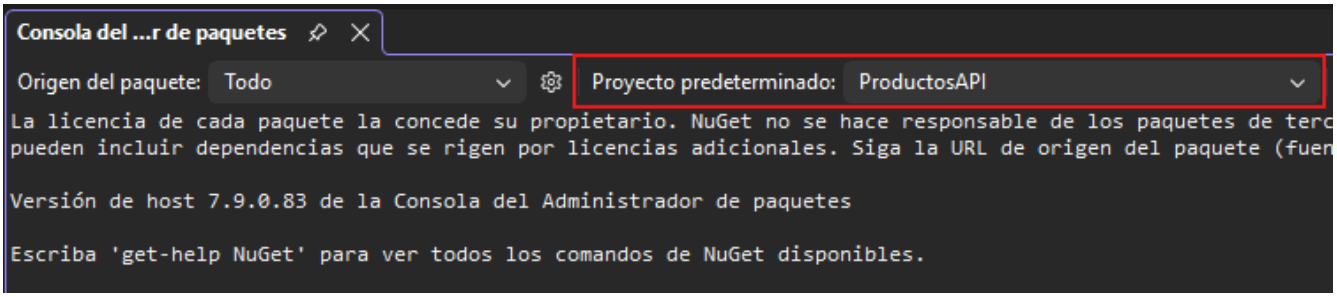
// Add services to the container.
builder.Services.AddDbContext<ProductosDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString( name: "DefaultConnection")));

builder.Services.AddControllers();
```

6. En la **Consola del Administrador de Paquetes** ejecute los comandos para generar una nueva migración y actualizar la base de datos. Asegúrese que en la consola este seleccionado por defecto el proyecto **ProductosAPI**.

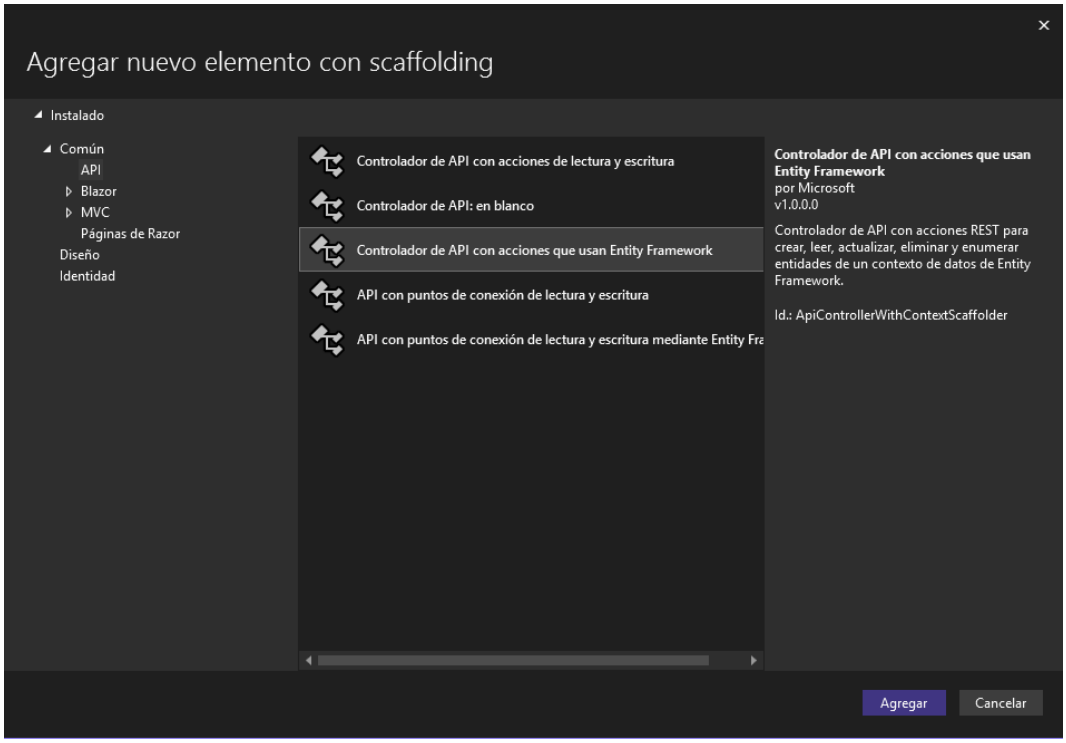
```
Add-Migration “Migración inicial”

Update-Database
```



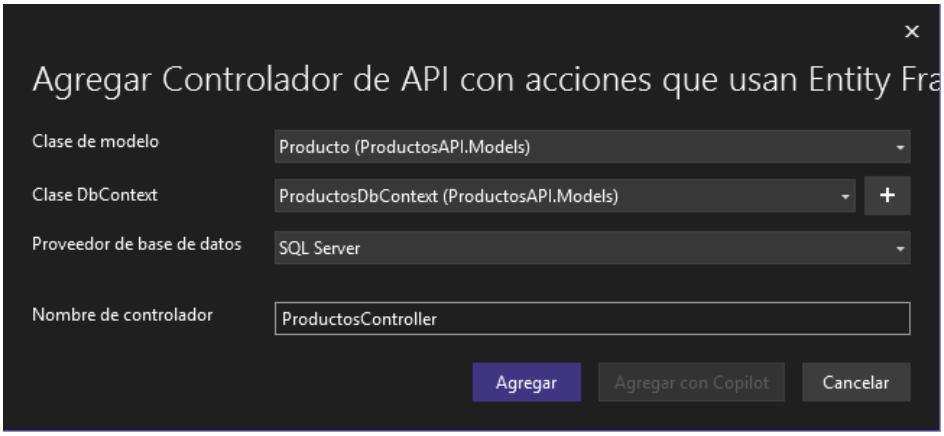
7. Agregue un nuevo controlador para los productos haciendo clic sobre la carpeta **Controllers** y seleccionando **Agregar > Controlador**.

8. Selecciona la categoría **API** y luego seleccione **Controlador API con acciones, usando Entity Framework**.

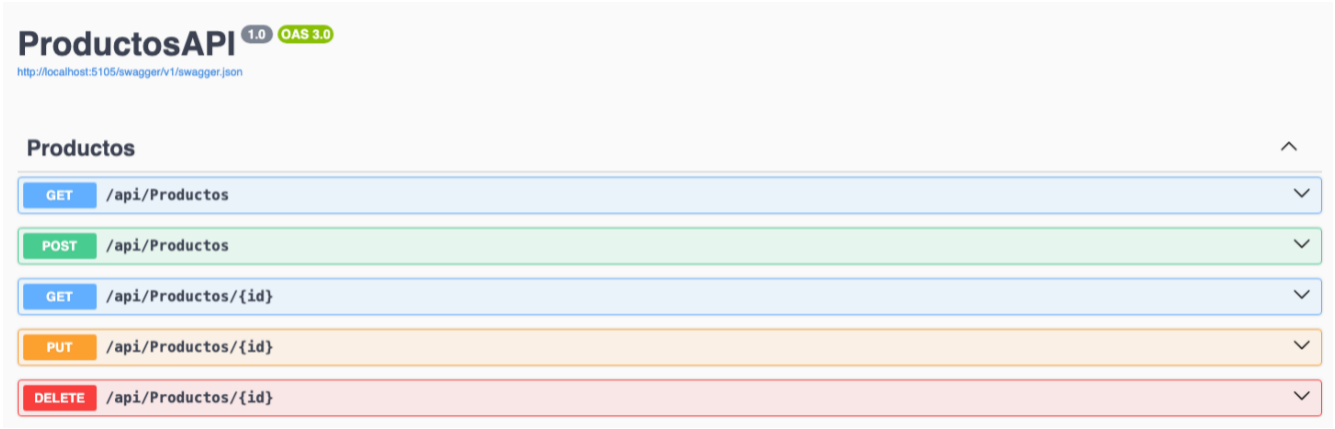


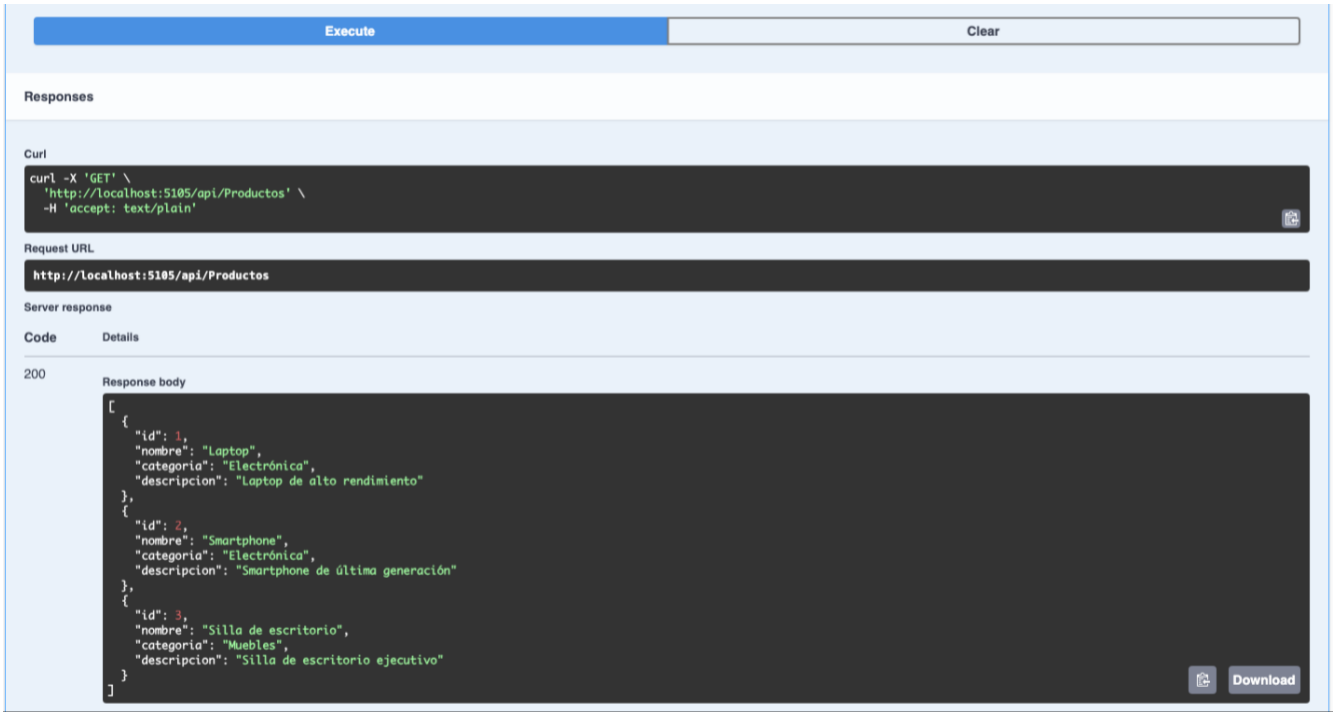
9. En la configuración del controlador, asegúrese de seleccionar las opciones como se muestra a continuación.

- **Clase de modelo**: Producto
- **Clase DbContext**: ProductoContext
- **Nombre del controlador**: ProductosController



10. Ejecute el proyecto y compruebe el funcionamiento, observe que los elementos fueron sembrados correctamente.





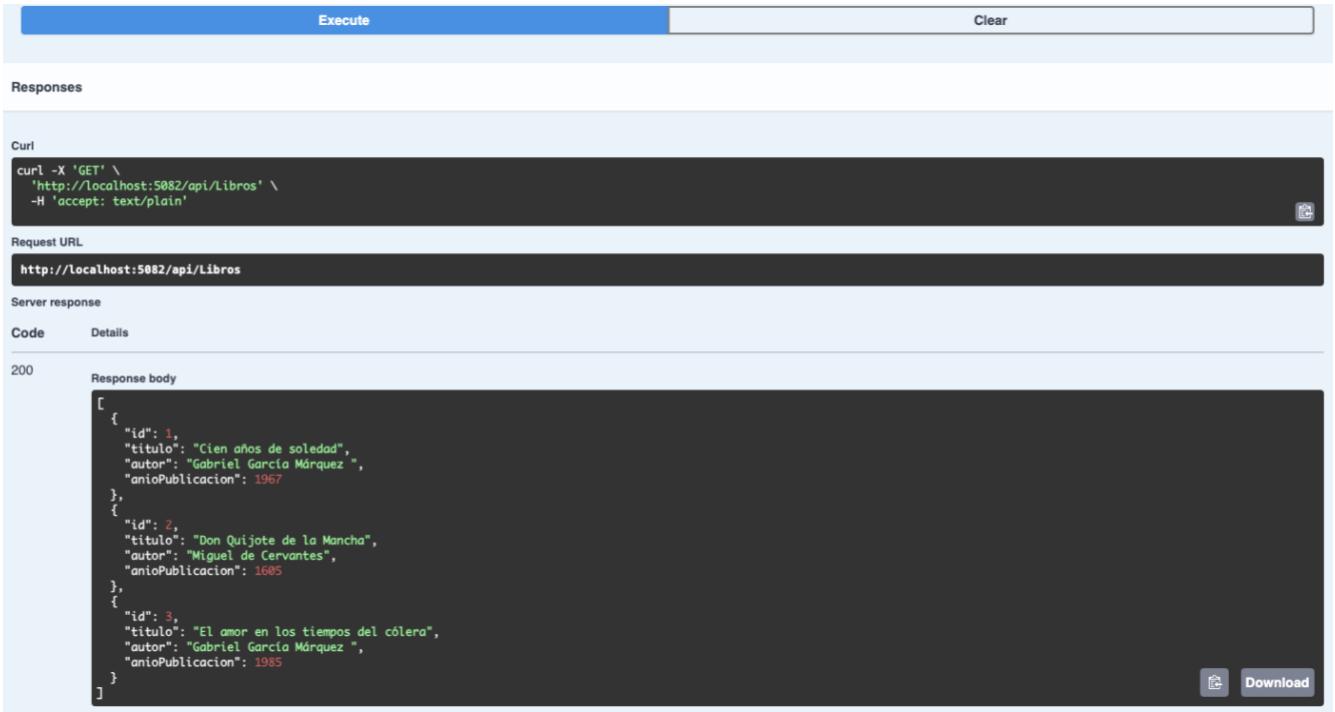
Parte 3 – API de libros

1. Realice en el proyecto **LibrosAPI** todos los pasos indicados en la **Parte 2**, adaptándolos al contexto de este nuevo proyecto. Para ello, deberá:

- Crear un modelo llamado Libro con los siguientes atributos:
    - Id
    - Titulo
    - Autor
    - AnioPublicacion
- Realizar las modificaciones que considere necesarias para adaptar los pasos de la **Parte 2** al modelo y contexto de **LibrosAPI**.
- Utilizar **DataSeed** para agregar los libros indicados a continuación:

Id	Titulo	Autor	Año publicación
1	Cien años de soledad	Gabriel García Marquez	1967
2	Don Quijote de la Mancha	Miguel de Cervantes	1605
3	El amor en los tiempos del cólera	Gabriel García Marquez	1985

2. Compruebe el funcionamiento de la API.



Parte 4 – Creación del API Gateway

1. Obtenga el puerto de ejecución de las API de productos y de libros, para ello puede ejecutar cada API y visualizar en el navegador el puerto por el cual se ejecuta cada API o ubique el archivo **properties > lauchSettings.json** visualice las **URLs** del perfil **http.**

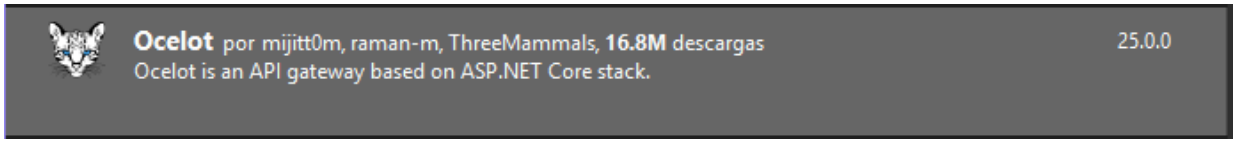
Productos API

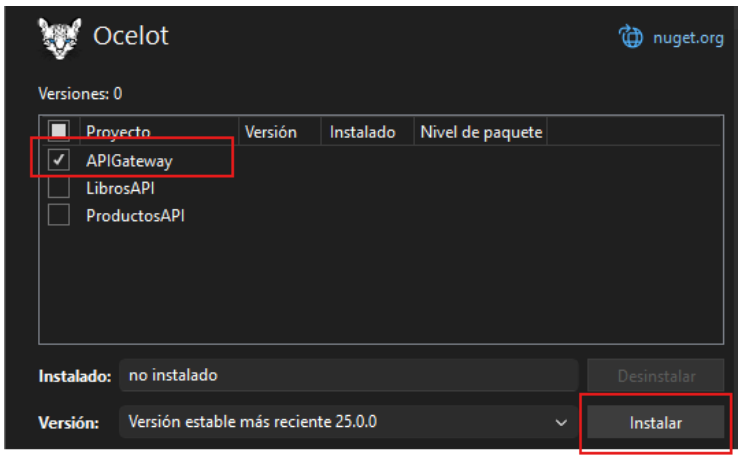
```
"http": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "launchUrl": "swagger",
  "applicationUrl": "http://localhost:5082",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
},
```

Libros API

```
"http": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "launchUrl": "swagger",
  "applicationUrl": "http://localhost:5105",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
},
```

2. En el proyecto **APIGateway** instale el paquete **Ocelot** usando el Administrador de Paquetes NuGet.





3. Abra la clase **Program** y modifique el contenido como se muestra a continuación.

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Configuration.AddJsonFile("ocelot.json", optional: false, reloadOnChange:
true);

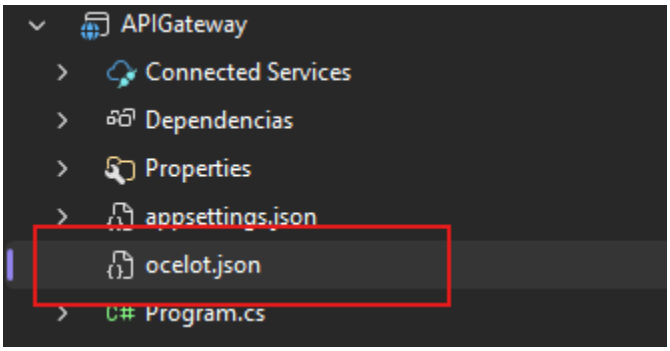
builder.Services.AddOcelot();

var app = builder.Build();

app.UseOcelot().Wait();

app.Run();
```

4. En la raíz del proyecto **APIGateway** cree un nuevo archivo con el nombre **ocelot.json.**



5. Abra el archivo **ocelot.json** y configure las rutas de la API de productos y libros de la siguiente forma.

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/productos",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5105
        }
      ],
      "UpstreamPathTemplate": "/productos",
      "UpstreamHttpMethod": [ "Get" ],
      "DownstreamHttpMethod": "Get",
      "SwaggerKey": "Productos"
    },
    {
      "DownstreamPathTemplate": "/api/productos/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5105
        }
      ],
      "UpstreamPathTemplate": "/productos/{id}",
```

```
      "UpstreamHttpMethod": [ "Get" ],
      "DownstreamHttpMethod": "Get",
      "SwaggerKey": "Productos"
    },
    {
      "DownstreamPathTemplate": "/api/productos",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5105
        }
      ],
      "UpstreamPathTemplate": "/productos",
      "UpstreamHttpMethod": [ "Post" ],
      "DownstreamHttpMethod": "Post",
      "SwaggerKey": "Productos"
    },
    {
      "DownstreamPathTemplate": "/api/productos/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5105
        }
      ],
      "UpstreamPathTemplate": "/productos/{id}",
      "UpstreamHttpMethod": [ "Put" ],
      "DownstreamHttpMethod": "Put",
      "SwaggerKey": "Productos"
    },
    {
      "DownstreamPathTemplate": "/api/productos/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5105
        }
      ],
      "UpstreamPathTemplate": "/productos/{id}",
      "UpstreamHttpMethod": [ "Delete" ],
      "DownstreamHttpMethod": "Delete",
      "SwaggerKey": "Productos"
    },
    {
      "DownstreamPathTemplate": "/api/libros",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5082
        }
      ],
      "UpstreamPathTemplate": "/libros",
      "UpstreamHttpMethod": [ "Get" ],
      "DownstreamHttpMethod": "Get",
      "SwaggerKey": "Libros"
    },
    {
```

```
      "DownstreamPathTemplate": "/api/libros/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5082
        }
      ],
      "UpstreamPathTemplate": "/libros/{id}",
      "UpstreamHttpMethod": [ "Get" ],
      "DownstreamHttpMethod": "Get",
      "SwaggerKey": "Libros"
    },
    {
      "DownstreamPathTemplate": "/api/libros",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5082
        }
      ],
      "UpstreamPathTemplate": "/libros",
      "UpstreamHttpMethod": [ "Post" ],
      "DownstreamHttpMethod": "Post",
      "SwaggerKey": "Libros"
    },
    {
      "DownstreamPathTemplate": "/api/libros/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5082
        }
      ],
      "UpstreamPathTemplate": "/libros/{id}",
      "UpstreamHttpMethod": [ "Put" ],
      "DownstreamHttpMethod": "Put",
      "SwaggerKey": "Libros"
    },
    {
      "DownstreamPathTemplate": "/api/libros/{id}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5082
        }
      ],
      "UpstreamPathTemplate": "/libros/{id}",
      "UpstreamHttpMethod": [ "Delete" ],
      "DownstreamHttpMethod": "Delete",
      "SwaggerKey": "Libros"
    }
  ]
}
```

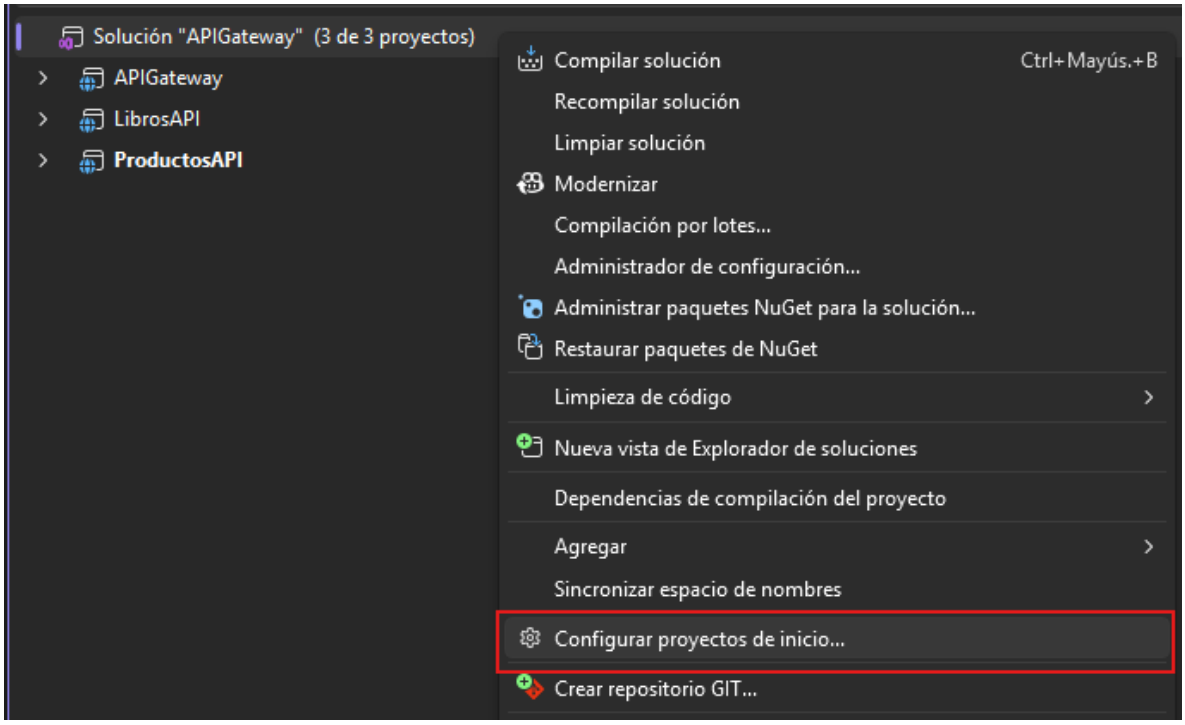
6. Después del arreglo de rutas, añada la sección **GlobalSettings** que corresponde a la configuración global del API Gateway.

```
{
  "Routes": [
  // RUTAS - NO COPIAR NUEVAMENTE
  ],
  "GlobalConfiguration": {
    "BaseUrl": "https://localhost:5017",
    "SwaggerEndPoints": [
      {
        "Key": "Productos",
        "Config": [
          {
            "Name": "Productos API",
            "Scheme": "http",
            "Host": "localhost",
            "Port": 5105,
            "Path": "/swagger/v1/swagger.json"
          }
        ]
      },
      {
        "Key": "Libros",
        "Config": [
          {
            "Name": "Libros API",
            "Scheme": "http",
            "Host": "localhost",
            "Port": 5082,
            "Path": "/swagger/v1/swagger.json"
          }
        ]
      }
    ]
  }
}
```

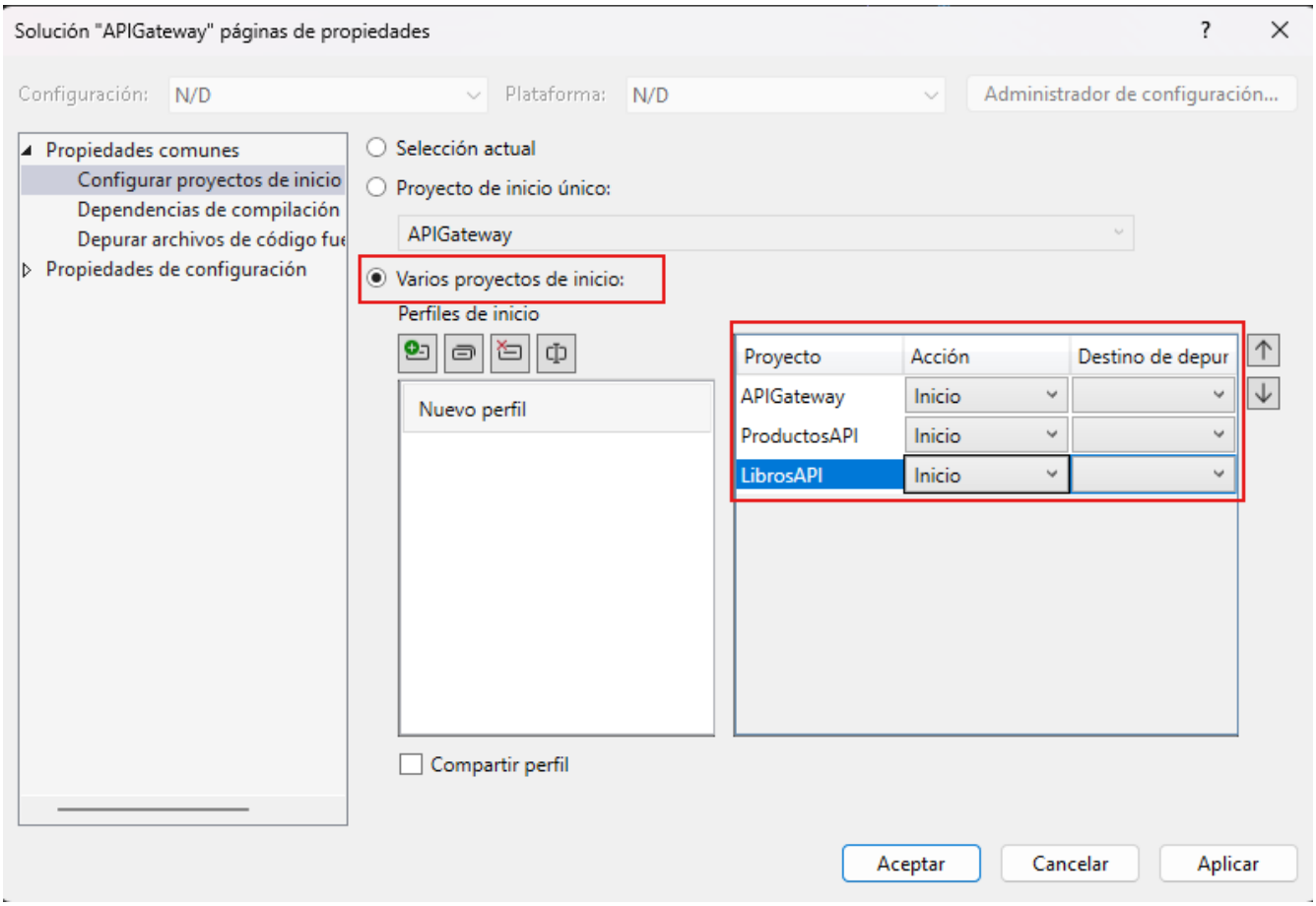
- En la propiedad **BaseUrl** utilice el puerto del perfil de ejecución **http** del proyecto **APIGateway**.

7. Guarde los cambios en el archivo.

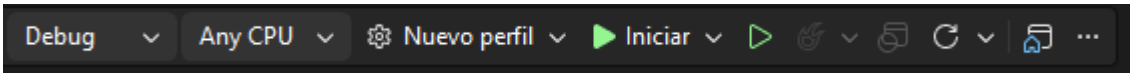
8. Haga clic derecho sobre la solución y seleccione la opción **Configurar proyectos de inicio**. Por defecto, Visual Studio ejecuta un proyecto a la vez por lo que cambiar esta configuración permitirá ejecutar los tres proyectos al mismo tiempo.



9. En la sección **Proyecto de inicio** seleccione la opción **Proyecto de inicio múltiples**. Luego, asegúrese que la acción de los tres proyectos sea **Iniciar**, finalmente haga clic en **Aceptar**.

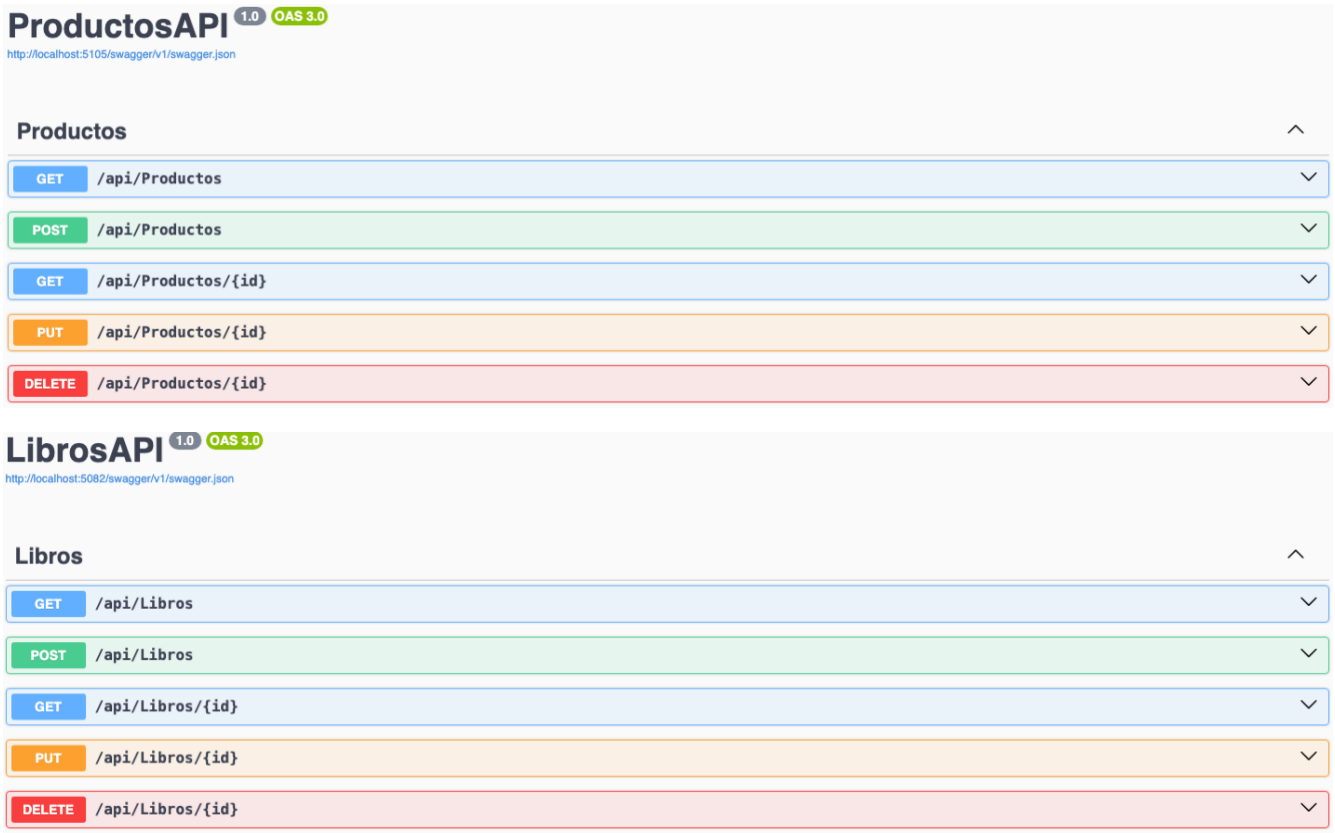


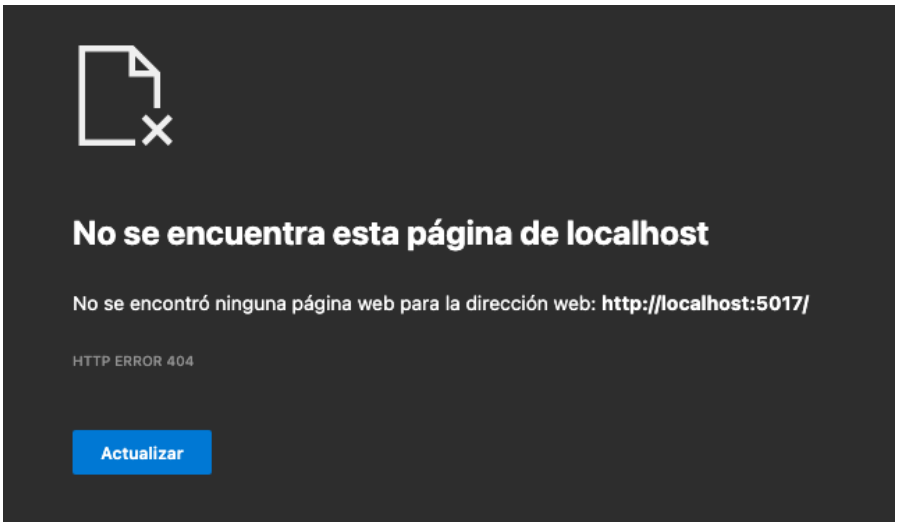
10. Haga clic en **Iniciar** para ejecutar todos los proyectos.



Podrá observar que en el navegador se abren tres pestañas, cada una correspondiente a un proyecto. En la ventana de **API Gateway** únicamente se muestra el mensaje **“No se encontró la página”**. Esto se debe a que Swagger aún no ha sido configurado.

Sin embargo, es posible realizar peticiones a la API mediante herramientas como **Postman**, **Thunder Client** o **Bruno**.





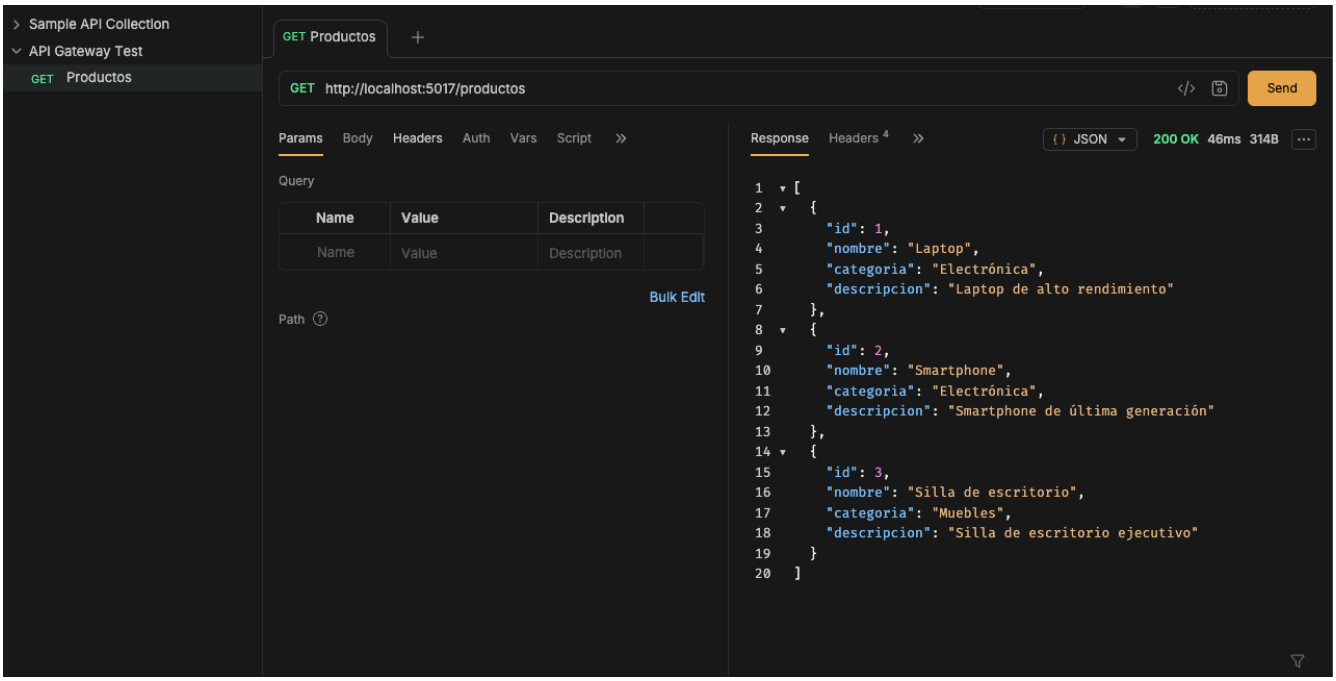
Las rutas para el API Gateway son las siguientes:

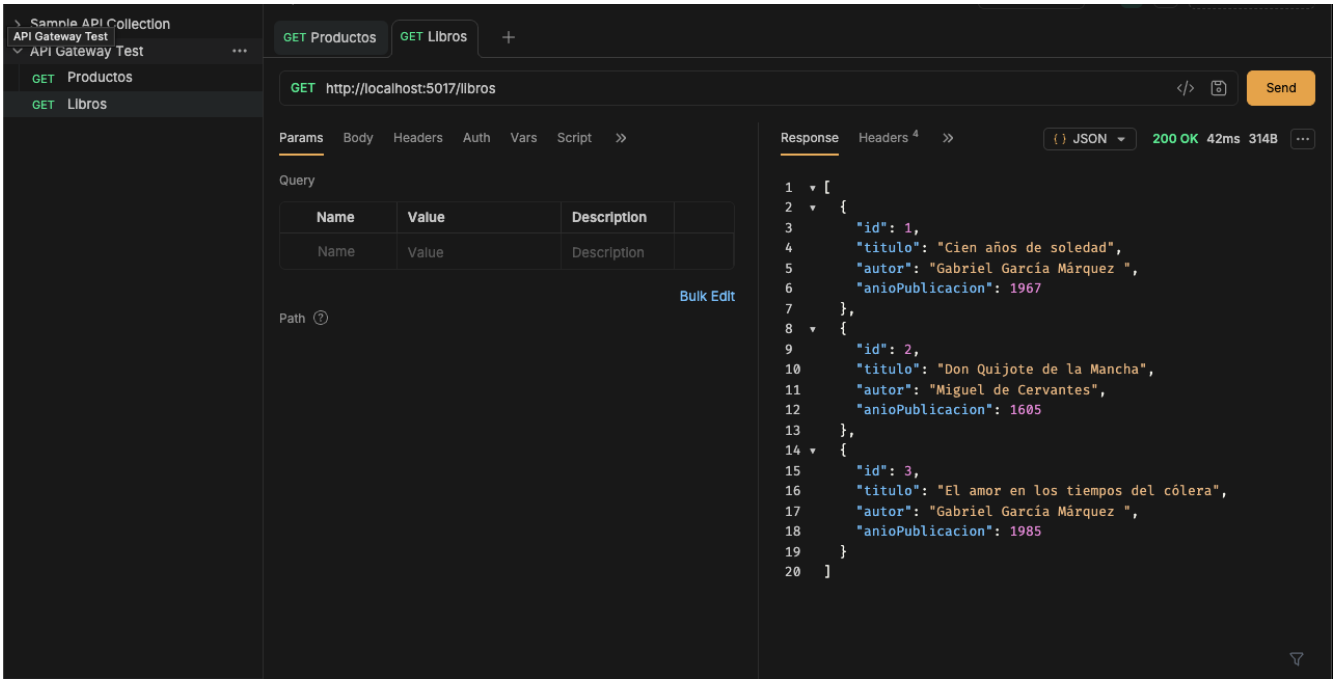
Productos:

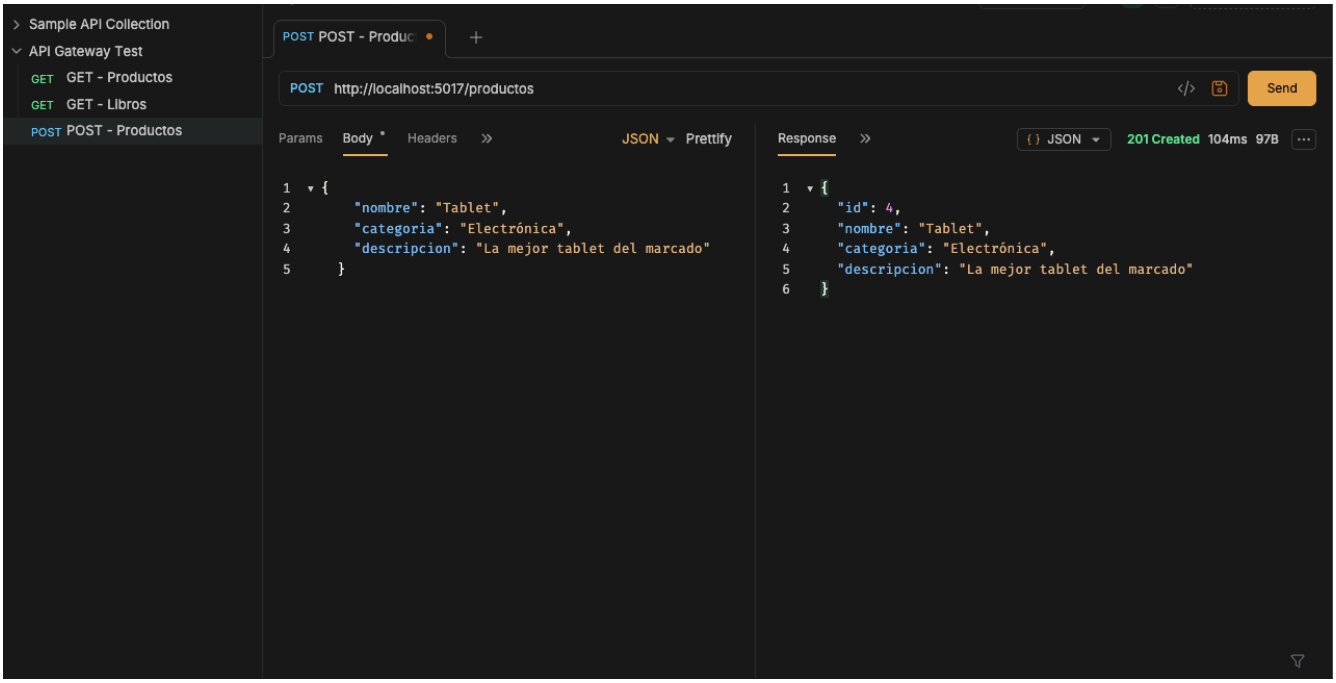
- **GET**: https://localhost:5017/productos
- **GET**: https://localhost:5017/productos/{id}
- **POST**: https://localhost:5017/productos
- **PUT**: https://localhost:5017/productos/{id}
- **DELETE**: https://localhost:5017/productos/{id}

Libros:

- **GET**: https://localhost:5017/libros
- **GET**: https://localhost:5017/libros/{id}
- **POST**: https://localhost:5017/libros
- **PUT**: https://localhost:5017/libros/{id}
- **DELETE**: https://localhost:5017/libros/{id}







Parte 5 – Configurando rate limit

La limitación de peticiones, o rate limiting, es una técnica utilizada para controlar la cantidad de solicitudes que un cliente puede hacer a un servicio en un periodo de tiempo específico. Esta técnica es crucial para proteger las aplicaciones de varios problemas, como sobrecarga del servidor, abusos, y ataques de denegación de servicio (DoS).

En Ocelot se puede configurar el rate limit de forma global, o especificarlo en cada uno de los métodos.

1. Para configurar la limitación de manera global, se debe agregar la sección **RateLimitOptions** en el archivo **ocelot.json**.

```
{
  "Routes": [ // RUTAS - NO COPIAR NUEVAMENTE ],
  "GlobalConfiguration": {
  "BaseUrl": "https://localhost:5017",
  "SwaggerEndPoints": [
    {
      "Key": "Productos",
      "Config": [
        {
          "Name": "Productos API",
          "Scheme": "http",
          "Host": "localhost",
          "Port": 5105,
          "Path": "/swagger/v1/swagger.json"
        }
      ]
    },
    {
      "Key": "Libros",
      "Config": [
        {
          "Name": "Libros API",
          "Scheme": "http",
          "Host": "localhost",
          "Port": 5082,
          "Path": "/swagger/v1/swagger.json"
        }
      ]
    }
  ],
  "RateLimitOptions": { // NUEVA CONFIGURACION
    "ClientIdHeader": "ClientId",
    "ClientWhitelist": [],
    "EnableRateLimiting": true,
    "Period": "1m",
    "PeriodTimespan": 60,
    "Limit": 100
  }
}
```

- Lo anterior hará que solo se puedan recibir 100 peticiones durante 1 minuto en cualquier endpoint, luego de llegar al máximo no se recibirán más peticiones. Modique el valor de la propiedad **Limit** a 10 y haga la prueba listando los productos.

2. Para aplicar el límite a un endpoint en específico, es necesario ubicar en el arreglo Routes el endpoint requerido, para este ejemplo **GET /productos**, modifique la ruta como se muestra a continuación.

```
"Routes": [
  {
    "DownstreamPathTemplate": "/api/productos",
    "DownstreamScheme": "http",
    "DownstreamHostAndPorts": [
      {
        "Host": "localhost",
        "Port": 5105
```

```
      }
    ],
    "UpstreamPathTemplate": "/productos",
    "UpstreamHttpMethod": [ "Get" ],
    "DownstreamHttpMethod": "Get",
    "SwaggerKey": "Productos",
    "RateLimitOptions": { // NUEVA CONFIGURACION
      "ClientIdHeader": "ClientId",
      "ClientWhitelist": [],
      "EnableRateLimiting": true,
      "Period": "1m",
      "PeriodTimespan": 60,
      "Limit": 5
    }
  },
```

- Esto indica que solo se pueden realizar 5 peticiones al endpoint en 60 segundos.



Parte 5 – Almacenamiento en cache

El almacenamiento en caché en Ocelot es una técnica que permite guardar temporalmente las respuestas de las solicitudes durante un período de tiempo determinado. Su objetivo es reducir la carga sobre los servicios backend y mejorar los tiempos de respuesta para los usuarios. Esta estrategia resulta especialmente útil cuando las respuestas de las solicitudes no cambian con frecuencia, ya que permite reutilizar la información almacenada en caché, optimizando el rendimiento y reduciendo el consumo de recursos.

1. Para configurar el almacenamiento en cache globalmente en Ocelot, agregue la sección **FileCacheOptions** en la configuración global del archivo **ocelot.json**.

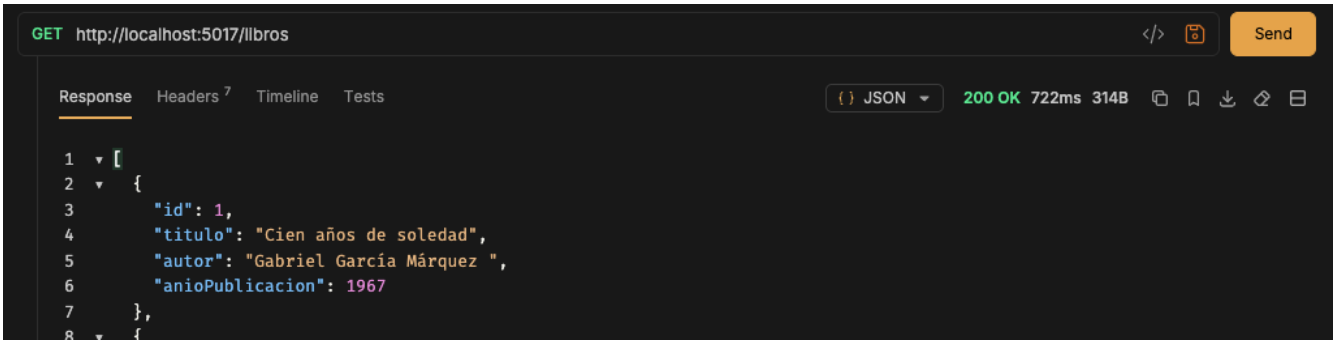
```
{
  "GlobalConfiguration": {
  "BaseUrl": "https://localhost:5017",
  "SwaggerEndPoints": [],
  "RateLimitOptions": {},
  "FileCacheOptions": { // NUEVA CONFIGURACION
    "ttlSeconds": 60,
    "Region": "default"
  }
}
```

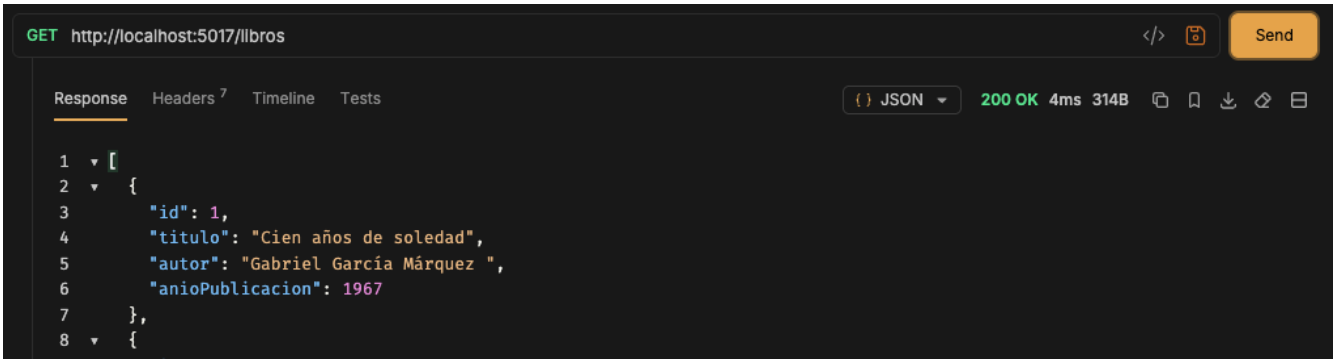
- Esto significa que, si un endpoint no ha sido solicitado anteriormente, su resultado se almacenará en caché. Luego, durante 60 segundos, si se realiza otra solicitud al mismo endpoint, se devolverá el resultado desde la caché en lugar de hacer una nueva llamada a la API.

2. Para aplicar el cache a un endpoint en específico, es necesario ubicar en el arreglo Routes el endpoint requerido, para este ejemplo **GET /libros**, modifique la ruta como se muestra a continuación.

```
{
  "DownstreamPathTemplate": "/api/libros",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "localhost",
      "Port": 5082
    }
  ],
  "UpstreamPathTemplate": "/libros",
  "UpstreamHttpMethod": [ "Get" ],
  "DownstreamHttpMethod": "Get",
  "SwaggerKey": "Libros",
  "FileCacheOptions": {
    "ttlSeconds": 5,
    "Region": "default"
  }
},
```

La primera petición al endpoint /libros se tardó 722 milisegundos, mientras que la segunda petición después de 5 segundo ya solo tardo 4 milisegundos.





V. DESARROLLO DE HABILIDADES

Ejercicio 1

Investigue cómo implementar autenticación en un API Gateway utilizando Ocelot y presente un ejemplo práctico de la configuración.